КОМП’ЮТЕРНИЙ ПРАКТИКУМ №4
Вивчення криптосистеми RSA та алгоритму електронного підпису; ознайомлення з методами генерації параметрів для асиметричних криптосистем

Мета роботи: Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.
Варіант: 8

Виконали: Вориняк Василь, Оніщук Максим ФБ-43.

Хід роботи

1. Написати функцію пошуку випадкового простого числа з заданого інтервалу або заданої довжини, використовуючи датчик випадкових чисел та тести перевірки на простоту. В якості датчика випадкових чисел використовуйте вбудований генератор псевдовипадкових чисел вашої мови програмування.

Розроблено модуль генерації великих простих чисел заданої бітової довжини. Генерація здійснюється за допомогою вбудованого генератора псевдовипадкових чисел, після чого кандидат проходить двоетапну перевірку:

1. Тест пробних ділень: швидке відсіювання чисел шляхом ділення на перші 100 простих чисел.
2. Тест Міллера-Рабіна: імовірнісний алгоритм. Обчислення модульної експоненти під час тесту здійснюється за допомогою алгоритму швидкого піднесення до степеня.

```
[        : …/Public/Crypta/lab4]$  python t1.py
[*] Починаємо генерацію 256-бітного простого числа...
[+] Знайдено просте число за 239 спроб

[*] Дебаг Міллера-Рабіна для підтвердженого простого числа:
58580330107751199370924234489958152443883787827777510733373182333556558939557
[*] Дебаг Міллера-Рабіна для числа p = 58580330107751199370924234489958152443883787827777510733373182333556558939557
[-] Розклад p-1: d = 14645082526937799842731058622489538110970946956944377683343295583389139734889, s = 2
[-] Обрано основу x = 88741143235569797280912059404395966987660984563062726802390779842663838029 02
[-] x_0 = (x^d mod p) = 1
[+] Основа пройшла перевірку: x^d mod p == 1 або p-1

[РЕЗУЛЬТАТ] Згенероване число p:
58580330107751199370924234489958152443883787827777510733373182333556558939557
```

2. За допомогою цієї функції згенерувати дві пари простих чисел p, q і p_1, q_1 довжини щонайменше 256 біт. При цьому пари чисел беруться так, щоб $pq \leq p_1q_1$; p і q - прості числа для побудови ключів абонента A, p_1 і q_1 - абонента B.

3. Написати функцію генерації ключових пар для RSA. Після генерування функція повинна повертати та/або зберігати секретний ключ (d, p, q) та відкритий ключ (n, e). За допомогою цієї функції побудувати схеми RSA для абонентів A і B - тобто, створити та зберегти для подальшого використання відкриті ключі (e, n), (e_1, n_1) та секретні d і d_1. Для однієї зі згенерованих пар ключів навести значення основних параметрів та перевірити виконання співвідношень, необхідних для коректної роботи RSA.

Для створення незалежних криптосистем абонентів A і B було згенеровано дві пари випадкових простих чисел довжиною 256 біт: p, q для A та p1, q1 для B. Під час генерації програмно забезпечується виконання критичної умови для роботи протоколу конфіденційного розсилання ключів: модуль одержувача має бути більшим або рівним модулю відправника. Якщо згенеровані пари не задовольняють цю умову, вони автоматично міняються місцями.

У якості відкритої експоненти *e* обрано просте число Ферма: 65537. Оскільки воно має лише дві одиниці у двійковому записі, що дозволяє виконувати піднесення до степеня з мінімальною кількістю операцій та пришвидшує роботу алгоритму. Крім того, простота числа забезпечує його взаємну простоту з функцією Ейлера, що необхідно для коректної роботи RSA.

```
[          : …/Public/Crypta/lab4]$  python t23.py
========================================
Завдання 2. ГЕНЕРАЦІЯ ПРОСТИХ ЧИСЕЛ ТА ПАРАМЕТРІВ
========================================

[+] Згенеровано просте число p = 906864666784422596499611358606376937562501940687990558944571322599347019463 87
[-] Приклади кандидатів, що не пройшли перевірку на простоту:
    Кандидат 1: 858830622562013820128081055713949206279167834079459210651732831660328646591309
    Кандидат 2: 110495384529128756640183946544417807295099272973582643822012784083029485525905
    Кандидат 3: 113415651561359956491814818563821306389380956193910655097674559181058184035945

[*] Дебаг Міллера-Рабіна для підтвердженого простого числа:
906864666784422596499611358606376937562501940687990558944571322599347019463 87
[*] Дебаг Міллера-Рабіна для числа p = 906864666784422596499611358606376937562501940687990558944571322599347019463 87
[-] Розклад p-1: d = 45343233339221129824980567930318846878125097034399527947228566129967350973193, s = 1
[-] Обрано основу x = 88842352149414092058230281628008832219014748834552326146389227183098289104 02
[-] x_0 = (x^d mod p) = 1
[+] Основа пройшла перевірку: x^d mod p == 1 або p-1

[+] Згенеровано просте число q = 837460343136315256355296546854633275120950799934478923614706471104195305753 03
[-] Приклади кандидатів, що не пройшли перевірку на простоту:
    Кандидат 1: 755416549738140608231698136886826232323841846794639149659236009519549502544 27
    Кандидат 2: 728158781111612349007017798311298002026514892948394558152549934422991163531795
    Кандидат 3: 976512165203337059890454686180853616919561131847069899817255033729183553661 17

[+] Згенеровано просте число p1 = 107299819081410224817616990008568468751915818848829978537783333880986578125029
[-] Приклади кандидатів, що не пройшли перевірку на простоту:
    Кандидат 1: 973911244819265012880068107053914707885800999933075493279951437816177017 01997597
    Кандидат 2: 108552372615213181803637839940083762488424357649449947981976821696469732352371
    Кандидат 3: 108137161160791081748069157551740130265067640346842832862201176615526011878455

[+] Згенеровано просте число q1 = 614096076785021069618930065340479052060459922938840446348895803299127932868 57
[-] Приклади кандидатів, що не пройшли перевірку на простоту:
    Кандидат 1: 757401542347283568456283208460368946935961324688311975572504075410727352743 29
    Кандидат 2: 688070774884257577451096486250346778812117693260334658683750406710937681365 33
    Кандидат 3: 114034008088361899177613846814943430557833392174002111686299807149061609711 71

[!] n_A > n_B. Міняємо ключі місцями для коректної роботи протоколу розсилання ключів.
```

```
========================================
Завдання 3. ГЕНЕРАЦІЯ КЛЮЧІВ
========================================

[АБОНЕНТ A]
Відкритий ключ (e, n):
  e = 65537
  n = 658923979376365623519827459841279339920985600698457013827429427686522716130449799282907221612427808451207360277
878465062177557165357468965918743720844385 3
Секретний ключ d:
  d = 587146841442559160204325556552955290793074402392366032828695602759319980722111511805740625779139008124506734431
037534481534610403131386810310419849256134 5

[АБОНЕНТ B]
Відкритий ключ (e1, n1):
  e1 = 65537
  n1 = 759463195023482744253794275372204653773422950228628780124043900913248178885844798179535950222506104853101627533
189983622946204590553673825363515719722802 61
Секретний ключ d1:
  d1 = 754109394939547730019922524220611530041129415233044202001497731845058901704307721165648351932066457756627701286
944444905340228653844817306728340852365517 3

========================================
ПЕРЕВІРКА СПІВВІДНОШЕНЬ
========================================
Перевірка (e * d) mod phi == 1 для Абонента A: 1
[+] Співвідношення виконується коректно! RSA математика налаштована правильно.
```

4. Написати програму шифрування, розшифрування і створення повідомлення з цифровим підписом для абонентів A і B.
5. За допомогою раніше написаних на попередніх етапах програм організувати роботу протоколу конфіденційного розсилання ключів з підтвердженням справжності по відкритому каналу за допомогою алгоритму RSA.

```
[ : …/Public/Crypta/lab4]$  python t45.py
[*] Генеруємо ключі для Абонентів...

[+] Згенеровано просте число p_A = 98383573714982646367131585795260012788479579689717318985209850628088670992339
[-] Приклади кандидатів, що не пройшли перевірку на простоту:
    Кандидат 1: 80731931675974793293802170938899888871559329752651349250285483885839221546745
    Кандидат 2: 60316707600598123389313683768019154616333628281527992693668570625317716733665
    Кандидат 3: 66603259288375680547790754877064544295903185511041460804539298420073632223799

[+] Згенеровано просте число q_A = 66382129129265953319100669609848778623555806423173988057952427641514133352263
[-] Приклади кандидатів, що не пройшли перевірку на простоту:
    Кандидат 1: 87830959636530057762736214862637945728362415419608716498887068228705495687401
    Кандидат 2: 83259902717486705507504678918192342650008295322893929120433604822371689650321
    Кандидат 3: 72929037433478528135470678041381926025006210169530755966237897916648196213597

[+] Згенеровано просте число p_B = 73630733867685461142956470347824398166045249425643804358771791329183680113221
[-] Приклади кандидатів, що не пройшли перевірку на простоту:
    Кандидат 1: 92992138077929977424345503072819352640744075458199719266955223686480484744323
    Кандидат 2: 90721292684238273027359019192944113401703017004846444342351792344377423052825
    Кандидат 3: 93268725207498687874598600323362729843444551924781529537244907177357129904329

[+] Згенеровано просте число q_B = 64134305549405088395599224061349239353146314781305244807746832417489042413841
[-] Приклади кандидатів, що не пройшли перевірку на простоту:
    Кандидат 1: 95826604169894034774716846656127070623314424270603609679127762408726302298793
    Кандидат 2: 66440102523605768843862418309742511186334470104849920640984313430837318550861
    Кандидат 3: 69291446298277492070841019323773953254907449893125994123681753308764335540269
```

```
==========================================
Завдання 4. Шифрування та Цифровий підпис
==========================================
Відкритий текст: 'CryptaLab4VOfb43'
Текст у вигляді числа (M): 896526606405964649239765537730520812O3

[ШИФРУВАННЯ] Криптограма (C), зашифрована ключем B:
402860567042774129995300445399494663204310061568183553270891118835296619181147890913342582092144622896326214939984391499922383776518316464526955288305689O

[РОЗШИФРУВАННЯ] Отримане число: 896526606405964649239765537730520812O3
Розшифрований текст: 'CryptaLab4VOfb43'

--- Цифровий підпис ---
[ПІДПИС] Цифровий підпис (S), створений ключем A:
367701463011053788292236644709587678235541979952700053124309033854550913802561999066838228320783618711498838615125246873439071042132505777209716996900874

[ПЕРЕВІРКА] Результат перевірки підпису Абонентом B: True

==========================================
Завдання 5. Протокол розсилання ключів
==========================================
Згенерований секретний симетричний ключ (k) для передачі:
1153318318915127119337304998623897281883788055947507660462672288216929690584555213126795786460062432165145747534860575558658161218609609254607731684726 37

Відправка від A до B...
Зашифрований ключ (k1) для передачі по каналу:
3102117948277815087098538909197643191925717990733427677222025532088063183714271414197794057334388359950053640073426151610455711182372015301075500507424523

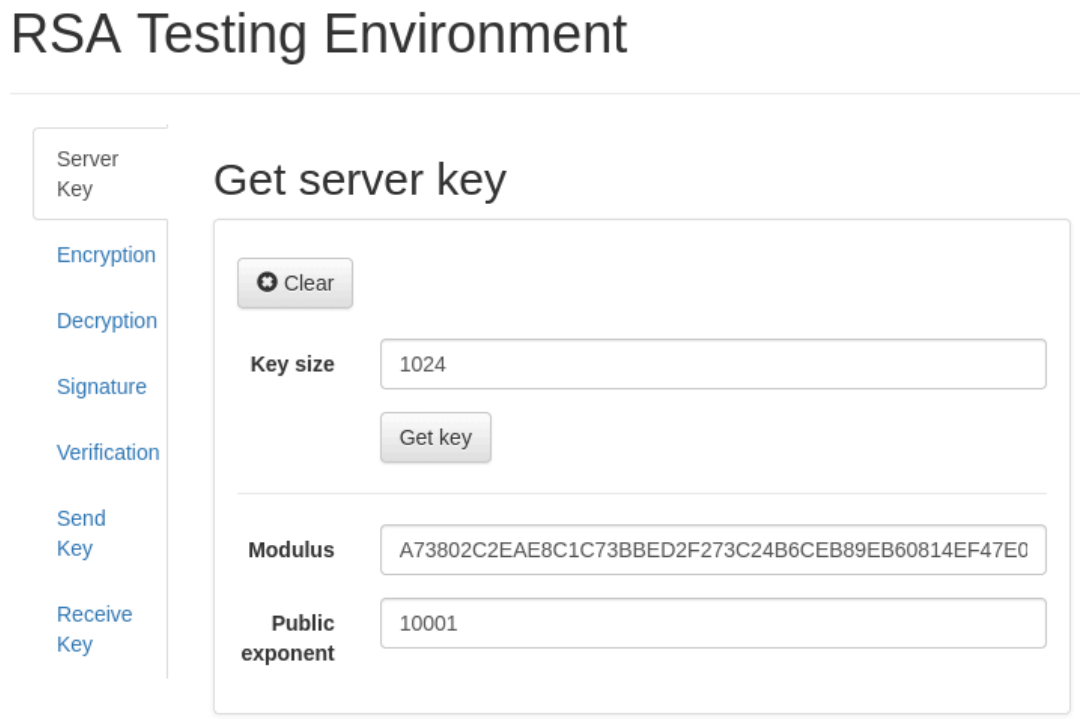
Зашифрований підпис (S1) для передачі по каналу:
6177092639610456624329645464639196319507781292402321721853727911251535908131674456571941304237905398085519657444150504270850551302589441981711278675323473

Отримання Абонентом B...
Розшифрований симетричний ключ:
1153318318915127119337304998623897281883788055947507660462672288216929690584555213126795786460062432165145747534860575558658161218609609254607731684726 37

Автентичність підтверджено: True
```

Автентифікація забезпечується шляхом створення цифрового підпису *S=k^d mod n* з використанням секретного ключа відправника. А конфіденційність забезпечується шляхом шифрування самого ключа *k* та підпису *S* відкритим ключем одержувача (*e1, n1)*. Розшифрувати ці дані може лише одержувач за допомогою свого секретного ключа *d1*

Перевірка з тестовим середовищем (http://asymcryptwebservice.appspot.com/?section=rsa)
Для тесту було створено окремий скрипт.




```
[ : …/Public/Crypta/lab4]$  python web_test.py
[*] Генерація локальних ключів...
[*] Починаємо генерацію 256-бітного простого числа...
[+] Знайдено просте число за 91 спроб
[*] Починаємо генерацію 256-бітного простого числа...
[+] Знайдено просте число за 255 спроб

========================================
ЛОКАЛЬНІ КЛЮЧІ:
Modulus (Bytes):
6B6EB97F656D30F24ED734BA3D8B83DDE69111FB2C9EFBDF1B0846BE736D9372F6A7412DDFD3C2B5F71218E3DFDEA72EA15CB24
0D33848F4640F8681747BCA73
Public Exponent (Bytes):
010001
========================================

--- Локально шифруємо → Сервер розшифровує ---
Відкритий текст: 'TestDecryption'
Ciphertext (Bytes):
A06B8EE70B2CB766EBF98BAF542457F769B3FAE6D16D3AEA59C316CBA8CE3E93C85D5B06D1A284B54385558A79F6150334BAE4C
8CD3BE41BC6C6390214094D10FE6537602B67B9F9A96A35D0EEDE868BD147C1D1DBD5804C645C52842CAEDDBDA2F78AE5C46AF7
E5811C9BAC3087234805442BA238C0C1227556C74B09461E95
```

Decryption

Clear

Ciphertext 7234805442BA238C0C1227556C74B09461E95 Text

Decrypt

Message TestDecryption

```
--- Сервер шифрує → Локально розшифровуємо ---
> Ciphertext з сайту: 243BA77494E499E4A8546A82B2C13F59C3F7DC9A64EB2DF110F5C8A10309CC966349E17146C259FBE
25DDB19FD663D5EB43F7E9ED750EF1D8F1ED59BEDB11919
Розшифрований текст: 'CryptaLab4VOfb43'
```

Encryption

Clear

Modulus B5F71218E3DFDEA72EA15CB240D33848F4640F8681747BCA73

Public exponent 10001

Message CryptaLab4VOfb43 Text

Encrypt

Ciphertext 243BA77494E499E4A8546A82B2C13F59C3F7DC9A64EB2DF110

```
--- Локально підписуємо → Сервер перевіряє ---
Message (Text): 'TestVerify'
Signature (Bytes):
1BACEECA4D265D2746ECE7833A43D7E5447DF020573A9293438F5076C98E58D69B8AC6D7ACDC808213CA6399A2373E6661AB781
7FC76F4236E63693863799BC5
```

Verify

Clear

Message TestVerify Text

Signature 1BACEECA4D265D2746ECE7833A43D7E5447DF020573A92934

Modulus B5F71218E3DFDEA72EA15CB240D33848F4640F8681747BCA73

Public exponent 10001

Verify

Verification true ✔

```
--- Сервер підписує → Локально перевіряємо ---
> Текст з сайту: SuperText
> Signature з сайту: 29590AC2711E5725CD10E80EC50B807AE76C19B562BF197CE67CC5B98BDCF54CBB2265937B75007767
6DF83266FDCA94C3E18C3DFF998B069C164F2E58DE6A79645A392405C6A69A01B537B2989F6A5D7124A15C39E73F64B51B847E2
48AAAAC4022B72DE6846E55556F837E2082E2ABB98C77C14326DC33C4381C2A664604F3
Результат перевірки підпису сервера: True
```

Sign

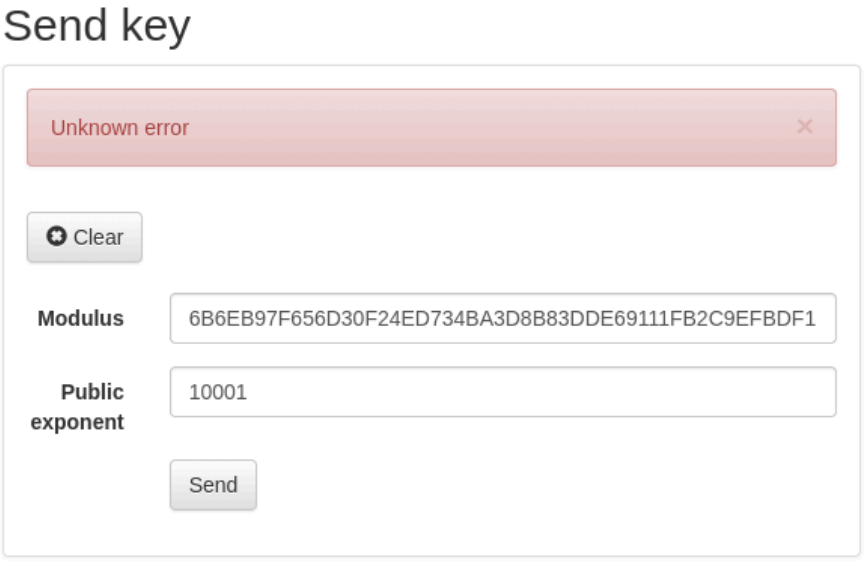
Clear

Message SuperText Text

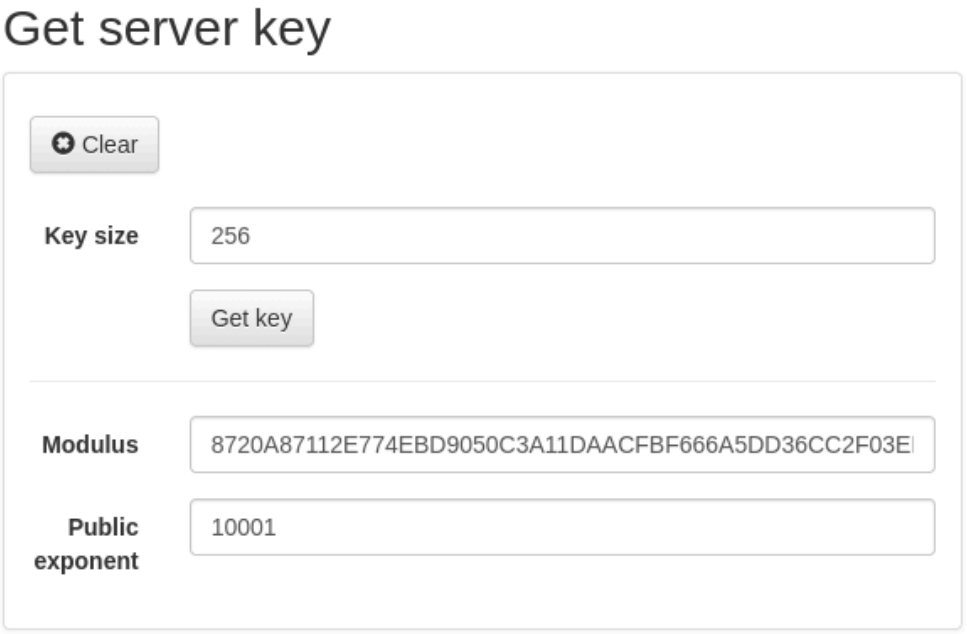
Sign

Signature 29590AC2711E5725CD10E80EC50B807AE76C19B562BF197CE6

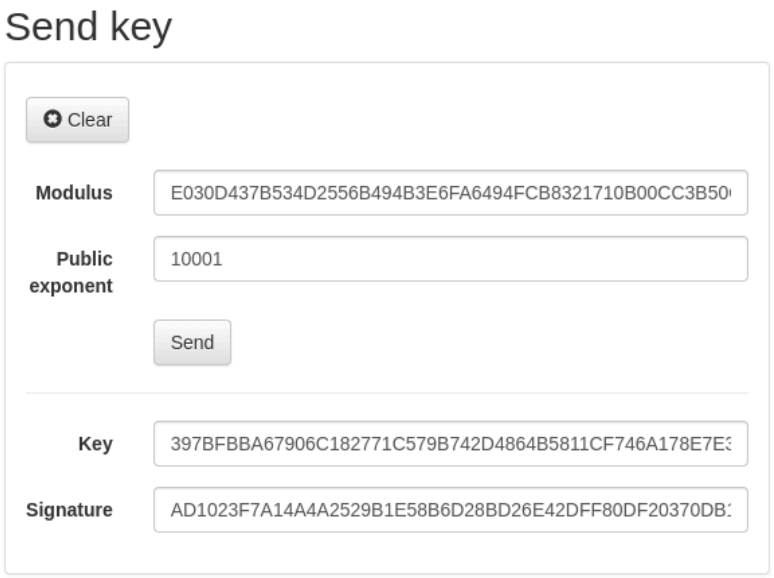
При тестуванні відправки ключа отримували наступну помилку:



Спробували використати меньший серверний ключ (256):




```
========================================
ЛОКАЛЬНІ КЛЮЧІ:
Modulus (Bytes):
E030D437B534D2556B494B3E6FA6494FCB8321710B00CC3B50C9E54A0E1A08CEBFB71262B3840C2DE7E798B591DE5B8CFEEA355
25688823E5519C37790C684DF
Public Exponent (Bytes):
010001
========================================
```



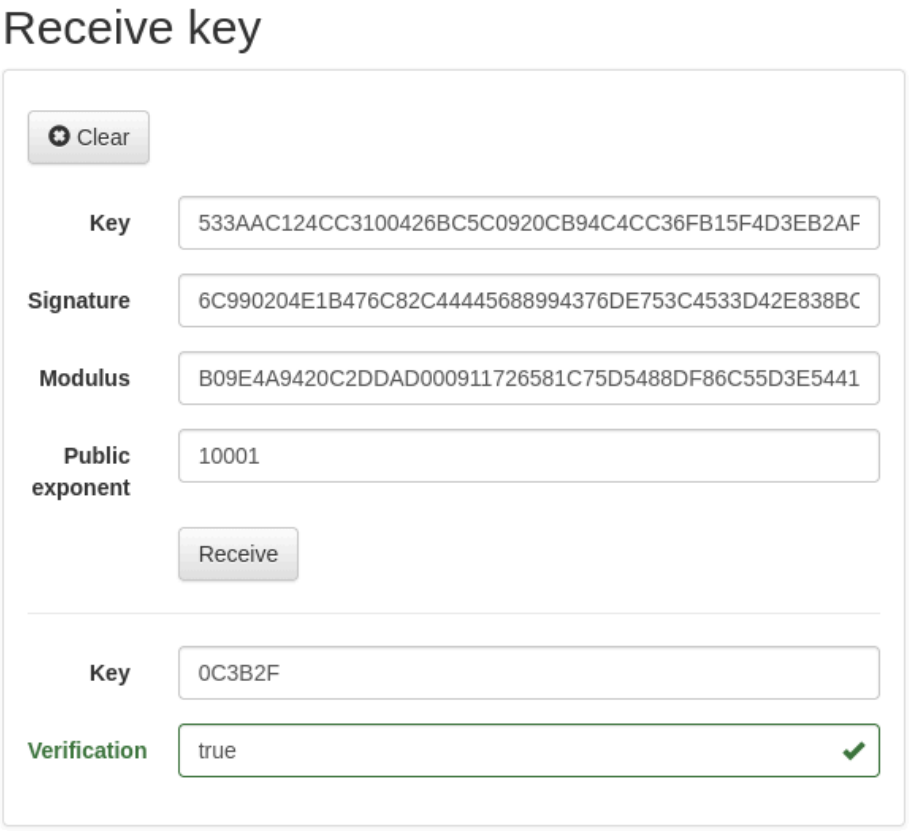

```
--- Сервер відправляє ключ → Локально отримуємо ---
> Key з сайту: 397BFBBA67906C182771C579B742D4864B5811CF746A178E7E36E359573EA792F0393942C35B4622A2DD958B
E707F5E79023154D7D6542F0A9411C7D4BBE776A
> Signature з сайту: AD1023F7A14A4A2529B1E58B6D28BD26E42DFF80DF20370DB1A59F3FA2933A293AFB867EAA30C86B7C
2F5C79F9498776B7735B2520BABA74359AC7255607E519
Секретний ключ сервера розшифровано:
8695379089567477425
Автентичність сервера: True
```

При використанні старого ключа розміром 1024 біти відправка не працювала, оскільки модуль відправника (сервера) був більше за модуль отримувача. Підпис не міг бути коректно зашифрований без втрати даних. Після генерації нового ключа розміром 256 біт відправка почала працювати, оскільки модуль сервера став меньшим за локальний модуль абонента. Це дозволило передати підпис без втрати його значення.

Отримання ключа перевіряється з ключем стандартної довжини:

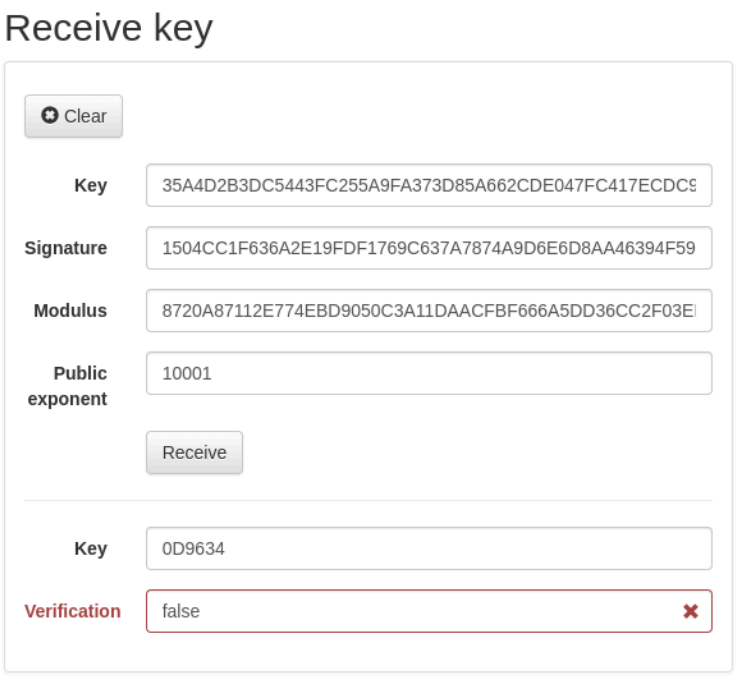
```
==========================================
ЛОКАЛЬНІ КЛЮЧІ:
Modulus (Bytes):
B09E4A9420C2DDAD000911726581C75D5488DF86C55D3E54413B26AE8B01B2CF69FA7CC0E43C84E6C2755F9FB87C2E5F84EFB5A
B2A08CE0F3AA141990B5A8397
Public Exponent (Bytes):
010001
==========================================
```

```
--- Локально відправляємо ключ → Сервер отримує ---
Key (Bytes):
533AAC124CC3100426BC5C0920CB94C4CC36FB15F4D3EB2AF0BA42FB19AE180041EFFEA9444BEC01508DDB1178F948C0973AF31
51C8DB1D45CC0D32B6D19BDC0DFA40538A0733425B63975AC6D6FA002311DBA99B6425BFF6263E45C709377D6372E43B02EC566
1E755C83C0D1A266DBB282492A54559AF24B715EA0C5F78540
Signature (Bytes):
6C990204E1B476C82C44445688994376DE753C4533D42E838BC016FF87824480F1AAD87E2043007F960C4BECA7251A982C1338D
63992AE45485E87C836A8C9CECE3734AD92AB56FC514EBBE873659DC13A47CD5E0ACDB75C0F13B9DA0E842DE63A3C90E76FFE56
297C788496633BFCCB220EEB45F220754E1C0EDE2FDA693AEA
```



Та коротким ключем:

```
--- Локально відправляємо ключ → Сервер отримує ---
Key (Bytes):
35A4D2B3DC5443FC255A9FA373D85A662CDE047FC417ECDC95B88B37B36E8D62
Signature (Bytes):
1504CC1F636A2E19FDF1769C637A7874A9D6E6D8AA46394F5978DAC29C122B8A
```



При спробі отримати ключ від сервера з тими ж параметрами, отримували *False.* Це відбувається через математичне переповнення модуля: сервер формував підпис розміром 1024 біти і намагався зашифрувати його 512-бітним локальним модулем абонента. При операції за меншим модулем відбувається безповоротна втрата бітів підпису сервера. Тому після розшифрування отримували пошкоджений підпис, що призводило до помилки верифікації. При зміні серверного модуля до 1024 біт, передавання працює коректно.

Контрольні питання

1. Які перевірки виконуються програмою під час генерації простих чисел і для чого потрібна кожна з них?

Виконуються дві перевірки. Тест пробних ділень на перші малі прості числа, який потрібен для оптимізації. Та імовірнісний тест Міллера-Рабіна, який шляхом перевірки сильної псевдопростоти гарантує, що число є простим з дуже високою ймовірністю.

2. Які умови повинні виконуватися для параметрів згенерованої пари ключів RSA? Як їх було перевірено у вашій реалізації?

p і *q* мають бути простими, *gcd(e, ф(n))* має дорівнювати 1, а також має виконуватися рівняння *e*d=1 mod(ф(n))*. У реалізації простота *p* і *q* перевіряється Міллера-Рабіном, *gcd* контролюється розширеним алгоритмом Евкліда, а виконання останньої умови явно перевіряється обчисленням *(e*d)%phi* з очікуваним результатом 1.

3. Які дані необхідні для виконання операцій Encrypt(), Decrypt(), Sign() та Verify()? Чому кожній із цих операцій не потрібні всі параметри RSA?

Encrypt() вимагає лише повідомлення та відкритий ключ адресата (*e, n*)
Decrypt() криптограму і секретний ключ одержувача (*d, n*).
Sign() повідомлення та секретний ключ відправника (*d, n*).
Verify() підпис, повідомлення і відкритий ключ відправника (*e, n*).
Всі параметри не потрібні для гарантування безпеки: операції шифрування та перевірки підпису є публічними і мають виконуватися лише з відкритим ключем, ізолюючи секретні дані.

4. Які властивості забезпечуються на різних етапах реалізованого протоколу передачі ключа: конфіденційність та підтвердження справжності відправника?

Конфіденційність забезпечується тим, що сам ключ та цифровий підпис шифруються відкритим ключем одержувача, тому розшифрувати їх може лише власник відповідного секретного ключа. Підтвердження справжності відправника забезпечується цифровим підписом, який створюється за допомогою секретного ключа відправника.

5. Які перевірки повинен виконати одержувач у реалізованому протоколі передачі ключа, перш ніж вважати отриманий ключ коректним і справжнім?

Одержувач спочатку повинен розшифрувати отримані дані за допомогою власного секретного ключа. Після цього він виконує операцію *Verify()*, використовуючи відкритий ключ відправника, щоб переконатися, що розшифрований підпис збігається з розшифрованим симетричним ключем. Тільки після успішної математичної перевірки підпису ключ вважається справжнім.

Висновок

У ході виконання лабораторної роботи було успішно розроблено програмну реалізацію асиметричної криптосистеми RSA. Практично закріплено методи генерації великих простих чисел з використанням тесту пробних ділень та тесту Міллера-Рабіна, реалізовано швидке піднесення до степеня за модулем за допомогою схеми Горнера та розширений алгоритм Евкліда для генерації ключів.

Усі криптографічні операції реалізовано з урахуванням вимог суворої ізоляції параметрів. Окрім цього, успішно імплементовано та протестовано протокол конфіденційного розсилання ключів із забезпеченням автентифікації відправника. Коректність роботи власної математичної реалізації підтверджено успішною взаємодією зі стороннім тестовим середовищем.

Також під час тестування було виявлено обмеження протоколу під час обміну ключами між абонентами з різними розмірами RSA модулів. Для усунення цієї проблеми можна використовувати геш-функцію: замість усього повідомлення підписувати його геш-код фіксованої довжини. Це дозволить уникнути залежності розміру підпису від довжини вихідного повідомлення.